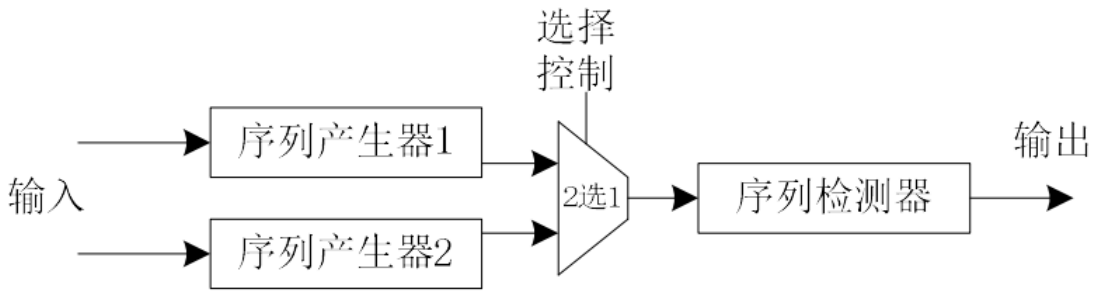
实验3实验报告

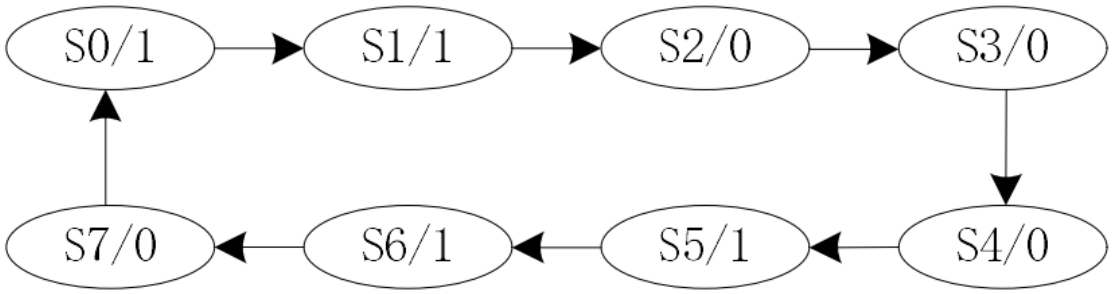
李毅PB22051031

1.实验设计

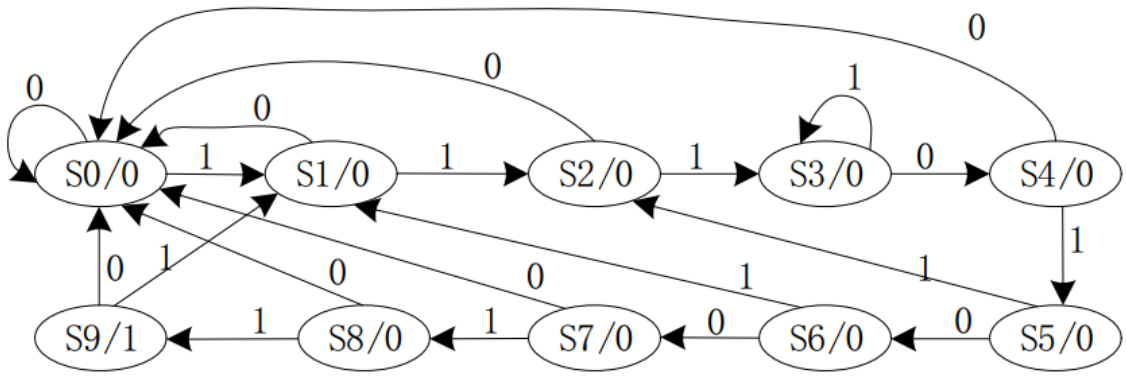


如图为本实验的设计思路。需要设计序列产生器，序列生成器，选择控制器与分频器等模块

下面是序列产生器状态转换图，这里序列产生器产生“11000110”序列：



下面是序列生成器状态转换图，这里序列检测器要检测“111010011”序列：



根据此状态图即可设计出模块。

2.代码

```
`timescale 10ns/1ns

module FPGA_EXP3 (
    input clk,
    input rst,
```

```
    input sel,
    output signal,
    output [3:0] debug_st
);
wire seq1;
wire seq2;
wire seq_in;
wire clk2;

div50MHZ div(
    .clk(clk),
    .rst(rst),
    .clkout(clk2)
);

seq_generator gen1(
    .clk(clk2),
    .rst(rst),
    .seq(seq1)
);

seq_generator_2 gen2(
    .clk(clk2),
    .rst(rst),
    .seq(seq2)
);

choose2_1 choose(
    .in1(seq1),
    .in2(seq2),
    .sel(sel),
    .out(seq_in)
);

seq_detector detector(
    .clk(clk2),
    .rst(rst),
    .seq_in(seq_in),
    .seq_out(signal),
    .debug_st(debug_st)
);

endmodule

module seq_generator (//序列产生器1
    input clk,
    input rst,
    output reg seq
);

reg [3:0] st;
parameter s0=0,s1=1,s2=2,s3=3,s4=4,s5=5,s6=6,s7=7,s8=8;
always @(posedge clk or negedge rst) begin
    if(!rst)
    begin
        st<=s0;
    end
    else begin
```

```
        case (st)
            s0:begin st<=s1;seq<=1'b1;end
            s1:begin st<=s2;seq<=1'b1;end
            s2:begin st<=s3;seq<=1'b0;end
            s3:begin st<=s4;seq<=1'b0;end
            s4:begin st<=s5;seq<=1'b0;end
            s5:begin st<=s6;seq<=1'b1;end
            s6:begin st<=s7;seq<=1'b1;end
            s7:begin st<=s8;seq<=1'b0;end
            s8:begin st<=s0;seq<=1'b0;end
            default: begin st<=s0;seq<=1'bx;end
        endcase
    end
end
endmodule

module seq_generator_2 (//序列产生器2
    input clk,
    input rst,
    output reg seq
);

reg [3:0] st;
parameter s0=0,s1=1,s2=2,s3=3,s4=4,s5=5,s6=6,s7=7,s8=8;
always @(posedge clk or negedge rst) begin
    if(!rst) begin
        st<=s0;
    end
    else begin
        case (st)
            s0:begin st<=s1;seq<=1'b1;end
            s1:begin st<=s2;seq<=1'b1;end
            s2:begin st<=s3;seq<=1'b1;end
            s3:begin st<=s4;seq<=1'b0;end
            s4:begin st<=s5;seq<=1'b1;end
            s5:begin st<=s6;seq<=1'b0;end
            s6:begin st<=s7;seq<=1'b0;end
            s7:begin st<=s8;seq<=1'b1;end
            s8:begin st<=s0;seq<=1'b1;end
            default: begin st<=s0;seq<=1'bx;end
        endcase
    end
end
endmodule

module choose2_1 (//2选1选择器
    input in1,
    input in2,
    input sel,
    output reg out
);

always @(*) begin
    case (sel)
        1'b0:out<=in1;
        1'b1:out<=in2;
        default: out<=1'bx;
    endcase
```

```
end
endmodule

module seq_detector(//序列检测器
    input clk,
    input rst,
    input seq_in,
    output reg seq_out,
    output reg [3:0] debug_st
);
reg [3:0] st;
parameter s0=0,s1=1,s2=2,s3=3,s4=4,s5=5,s6=6,s7=7,s8=8,s9=9;
always @(posedge clk or negedge rst) begin
    if(!rst) begin
        st<=s0;
    end
    else begin
        debug_st<=st;
        case (st)
            s0: begin
                    seq_out<=0;
                    if (seq_in==1'b0)
                        st<=s0;
                    else if(seq_in==1'b1)
                        st<=s1;
                    else
                        st<=s0;
                end
            s1: begin
                    seq_out<=0;
                    if (seq_in==1'b0)
                        st<=s0;
                    else if(seq_in==1'b1)
                        st<=s2;
                    else
                        st<=s0;
                end
            s2: begin
                    seq_out<=0;
                    if (seq_in==1'b0)
                        st<=s0;
                    else if(seq_in==1'b1)
                        st<=s3;
                    else
                        st<=s0;
                end
            s3: begin
                    seq_out<=0;
                    if (seq_in==1'b0)
                        st<=s4;
                    else if(seq_in==1'b1)
                        st<=s3;
                    else
                        st<=s0;
                end
            s4: begin
                    seq_out<=0;
                    if (seq_in==1'b0)
```

```
                    st<=s0;
                else if(seq_in==1'b1)
                    st<=s5;
                else
                    st<=s0;
            end
        s5: begin
                seq_out<=0;
                if (seq_in==1'b0)
                    st<=s6;
                else if(seq_in==1'b1)
                    st<=s2;
                else
                    st<=s0;
            end
        s6: begin
                seq_out<=0;
                if (seq_in==1'b0)
                    st<=s7;
                else if(seq_in==1'b1)
                    st<=s1;
                else
                    st<=s0;
                end
        s7: begin
                seq_out<=0;
                if (seq_in==1'b0)
                    st<=s0;
                else if(seq_in==1'b1)
                    st<=s8;
                else
                    st<=s0;
            end
        s8: begin
                seq_out<=0;
                if (seq_in==1'b0)
                    st<=s0;
                else if(seq_in==1'b1)
                    st<=s9;
                else
                    st<=s0;
            end
        s9: begin
                seq_out<=1;
                if (seq_in==1'b0)
                    st<=s0;
                else if(seq_in==1'b1)
                    st<=s1;
                else
                    st<=s0;
            end
        default:begin
                seq_out<=1'bx;
                st<=s0;
                end
    endcase
  end
end
```

```
endmodule

module div50MHZ(//分频器
    input clk,
    input rst,
    output reg clkout
);
    integer cnt;
    always @(posedge clk or negedge rst) begin
        if(!rst)
        begin
            cnt <= 0;
            clkout <= 1'b0;
        end
        else
        begin
            if(cnt <= 249999 && cnt>= 0)
                begin
                    clkout <= 1'b1;
                    cnt <= cnt + 1;
                end
            else if(cnt >=250000 && cnt <= 499999)
                begin
                    clkout <= 1'b0;
                    cnt <= cnt + 1;
                end
            else
                begin
                    cnt<=0;
                end
        end

    end
endmodule
```

3.实验结果

3.1 仿真波形

注：仿真时为了方便查看波形，未接分频器

激励文件：

```
`timescale 10ns/1ns

module FPGA_EXP3_tb();

reg clk;
reg rst;
reg sel;
wire signal;
wire[3:0] debug_st;

FPGA_EXP3 fpga(
    .clk(clk),
```

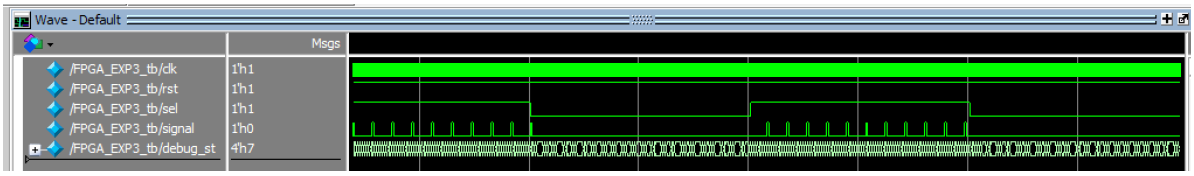
```
    .rst(rst),
    .sel(sel),
    .signal(signal),
    .debug_st(debug_st)
);

initial begin
   rst=0;
   #5 rst=1;
   #5 sel=1;
   #1000 sel=0;
   #1000 sel=1;
   #1000 sel=0;
   #1000 sel=1;
end

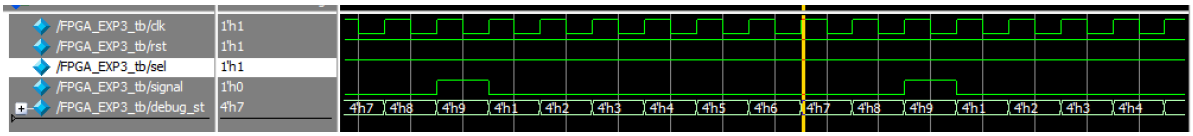
initial clk = 1'b0;
always #5 clk = ~clk; //产生时钟信号

endmodule
```

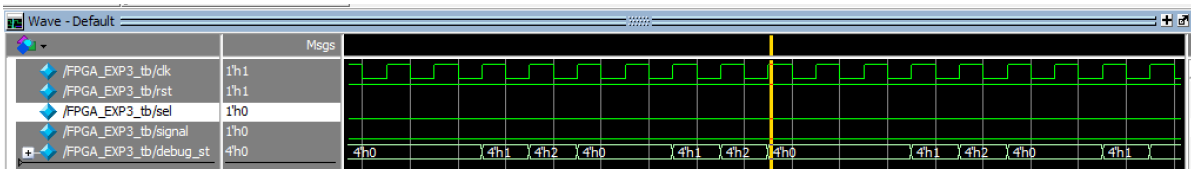
仿真波形



sel=1（目标序列时）



sel=0（非目标序列时）

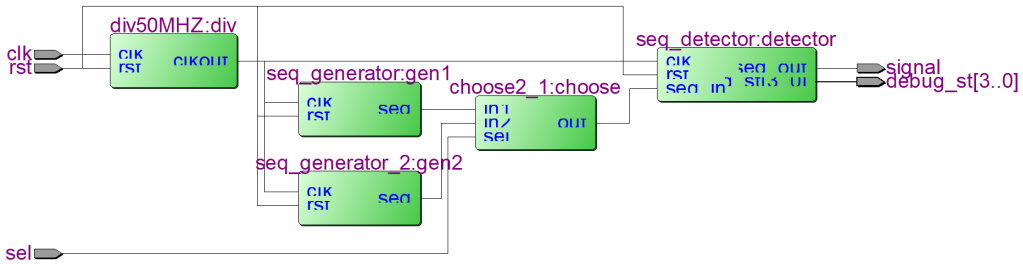


3.2 RTL电路

全编译之后的资源占用信息：

Flow Summary	
Flow Status	Successful - Mon Oct 28 14:08:17 2024
Quartus II 64-Bit Version	13.0.1 Build 232 06/12...3 SP 1 SJ Full Version
Revision Name	FPGA_EXP3
Top-level Entity Name	FPGA_EXP3
Family	Cyclone V
Device	5CEFA2F23C8
Timing Models	Final
Logic utilization (in ALMs)	12 / 9,430 (< 1 %)
Total registers	28
Total pins	8 / 224 (4 %)
Total virtual pins	0
Total block memory bits	0 / 1,802,240 (0 %)
Total DSP Blocks	0 / 25 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI PMA RX ATT Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total HSSI PMA TX ATT Serializers	0
Total PLLs	0 / 4 (0 %)
Total DLLs	0 / 4 (0 %)

RTL电路：



3.3 上板结果

管脚分配：

Node Name	Direction	Location	I/O Bank	VREF Group	...reserved	...Standard	...ent Stre	...lew Rat	...erential	...nalog Se	...VCCT_G	I/O Pin	...ted Ref	...non Mo	...Slew R	...erential	Commo
clk	Input	PIN_W16	4A	B4A_N0		2.5...lt)	12m...lt)										
debug_st[3]	Output					2.5...lt)	12m...lt)	1 (...lt)									
debug_st[2]	Output					2.5...lt)	12m...lt)	1 (...lt)									
debug_st[1]	Output					2.5...lt)	12m...lt)	1 (...lt)									
debug_st[0]	Output					2.5...lt)	12m...lt)	1 (...lt)									
rst	Input	PIN_R11	3B	B3B_N0		2.5...lt)	12m...lt)										
sel	Input	PIN_Y11	3B	B3B_N0		2.5...lt)	12m...lt)										
signal	Output	PIN_D12	7A	B7A_N0		2.5...lt)	12m...lt)	1 (...lt)									
<<new node>>																	

上板结果：

上板LED在sel=1时可以不断闪烁。即识别到了既定序列

可以正常刷新显示。

4.实验总结

本次实验对自动机的实现有了更深入的认识